AI & Generative AI Practice Knowledge Base

A practice PDF for the Phase 2 RAG application

Purpose: This document is designed specifically for the RAG program in Lesson 19. It contains enough structured information to test PDF extraction, chunking, Gemini embeddings, ChromaDB retrieval, prompt augmentation, grounded generation, and questions whose answers are absent from the document.

1. Artificial Intelligence

Artificial Intelligence (AI) is the field of building computer systems that perform tasks that normally require human intelligence. These tasks include understanding language, recognizing images, making predictions, solving problems, and generating content.

Machine learning is a major approach to AI. Instead of explicitly programming every rule, a machine-learning system learns patterns from examples. Deep learning is a subset of machine learning that uses neural networks with many layers.

AI applications can be grouped into areas such as computer vision, natural language processing, recommendation systems, robotics, speech processing, and generative AI.

2. Generative AI

Generative AI refers to AI systems that can create new content from learned patterns. The content may include text, images, audio, video, or computer code.

Large language models (LLMs) are generative AI systems specialized in language. They can summarize documents, answer questions, write code, transform text, and generate structured responses.

A generative model does not retrieve a guaranteed fact from a database every time it answers. It generates a response based on its learned patterns and the information supplied in the current context.

3. Large Language Models

An LLM processes text as tokens rather than as whole sentences in the way humans perceive them. Tokens can represent words, parts of words, punctuation, or other pieces of text.

During generation, an LLM predicts likely next tokens based on the preceding context. The model can produce fluent answers, but fluent language does not guarantee factual correctness.

Context windows define how much input and output context a model can process in a request. Very large documents can be inefficient or exceed a model's context capacity, which is one reason retrieval systems are useful.

4. Transformers and Attention

Transformers are neural-network architectures widely used for modern language models. A key component is attention, which allows the model to weigh relationships between tokens in a sequence.

Self-attention helps a model connect relevant words or phrases even when they are far apart in a passage. Transformer architectures enabled major improvements in language understanding and generation.

5. Embeddings

An embedding is a numerical vector representation of content. Texts with related meanings tend to have vectors that are close together in an embedding space.

Embeddings are useful for semantic search because a query such as 'How many days can an employee take off?' can retrieve a passage about annual leave even if the exact words in the query do not appear in the passage.

For a semantic-search system, documents and queries should normally be embedded using the same embedding model or a compatible embedding space.

6. Semantic Search

Semantic search retrieves information according to meaning rather than requiring exact keyword matches. It converts a query into an embedding and compares that vector with stored document embeddings.

Cosine similarity is a common way to compare embedding vectors. A higher similarity generally indicates that two vectors point in more similar directions.

Semantic search is especially useful when users express a concept differently from the wording used in the source documents.

7. Vector Databases

A vector database is designed to store and retrieve high-dimensional vectors efficiently. It can maintain vectors along with document text, IDs, and metadata.

FAISS is a similarity-search library that provides efficient vector indexing and search. ChromaDB is a developer-friendly vector database commonly used for local AI and RAG prototypes. Qdrant, Pinecone, and Weaviate are other popular vector-search systems.

Approximate nearest-neighbor methods can search large vector collections much faster than comparing a query against every vector. HNSW is one widely used indexing approach.

8. ChromaDB

ChromaDB organizes vector data into collections. A collection can contain IDs, documents, embeddings, and metadata.

Persistent ChromaDB storage allows a knowledge base to remain available after the Python process exits. Metadata can describe the source document, page number, section, category, or chunk ID.

Metadata filtering can restrict retrieval to documents or chunks that satisfy a condition, such as category equals 'AI' or source equals a particular PDF.

9. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, or RAG, combines retrieval with language generation. Instead of asking an LLM to answer only from its learned parameters, the system first retrieves relevant external information and places that information into the model's prompt.

A typical RAG flow is: ingest documents, extract text, split text into chunks, create embeddings, store vectors, embed the user query, retrieve the most relevant chunks, build a prompt containing the retrieved context, and generate an answer.

RAG does not retrain or modify the underlying LLM. Updating the knowledge base generally involves adding, updating, or deleting documents and their embeddings.

10. PDF Document Processing

A text-based PDF contains machine-readable text that libraries such as pypdf can often extract. A scanned PDF may contain page images instead of selectable text and may require OCR.

PDF processing should preserve useful document structure when possible, including page numbers, headings, sections, tables, and source information. Page-aware metadata is valuable for citations and debugging retrieval.

11. Chunking

Chunking means dividing a large document into smaller pieces before embedding. A single embedding for a long document can mix unrelated topics and make precise retrieval harder.

Fixed-size chunking divides text into pieces based on a target character or token size. Chunk overlap repeats some text between neighboring chunks so that information near a boundary retains context.

Paragraph-based or section-based chunking can preserve natural semantic boundaries. Recursive chunking can try larger boundaries first and split further when a chunk is too large.

A useful starting point for many text RAG systems is a moderate chunk size with modest overlap, but the best values depend on the document type, embedding model, and questions users ask.

12. Retrieval and Top-K

Top-K retrieval means returning the K most relevant chunks for a query. A small K can miss useful context, while a very large K can add irrelevant information and increase prompt size.

Retrieval quality should be evaluated independently from answer quality. If the correct passage is not retrieved, a strong LLM may still fail because it does not have the required evidence in its context.

Similarity scores, retrieved chunk IDs, source pages, and retrieved text should be logged during development so retrieval errors can be diagnosed.

13. Prompt Augmentation and Grounding

In a RAG system, the retrieved text is placed into a prompt as context. A good instruction tells the model to answer using the supplied context and to say when the context does not contain enough information.

Grounding means tying the answer to retrieved evidence. A grounded answer should not invent a fact that is absent from the supplied context.

Source attribution can improve trust. A production system can store page and document metadata and include source references in the final response.

14. Hallucinations

An AI hallucination is an output that contains information that is unsupported, incorrect, or fabricated. Retrieval can reduce hallucinations by supplying relevant evidence, but RAG does not guarantee perfect factuality.

Common causes of poor RAG answers include retrieving the wrong chunks, retrieving too little context, outdated documents, ambiguous questions, weak prompts, and model errors.

A robust system should allow the model to respond that there is insufficient information when the retrieved context does not support an answer.

15. RAG Production Architecture

A production RAG system usually separates ingestion from query-time answering. Ingestion can be run when documents are added or changed; query-time processing should only embed the user's question, retrieve relevant content, and generate the response.

A typical architecture is: data sources -> extraction -> chunking -> embeddings -> vector database. At query time: user question -> query embedding -> vector retrieval -> optional filtering or reranking -> prompt construction -> LLM -> grounded answer.

Production improvements can include batching, retry logic, rate-limit handling, embedding caches, metadata filters, hybrid retrieval, reranking, access control, source citations, document versioning, and evaluation.

16. Topics for Practice Questions

- Question: What is the difference between AI and generative AI?
- Question: Why are embeddings useful for semantic search?
- Question: What does RAG stand for?
- Question: Why does a RAG system split documents into chunks?
- Question: What is the role of ChromaDB in a RAG application?
- Question: What is cosine similarity used for?
- Question: Why should document and query embeddings use the same embedding space?
- Question: What is the difference between retrieval and generation?
- Question: What is chunk overlap and why is it useful?
- Question: What is a hallucination?
- Question: Why is retrieving too many chunks potentially harmful?
- Question: What is the difference between ingestion time and query time in a RAG system?
- Question: Which technology in this document is used for storing and searching vectors?

- Question: Which component generates the final natural-language answer?
- Question: What information can metadata contain in a RAG system?

17. Questions Not Answered by This Document

The exact annual leave policy of a specific company.

The current weather in Thiruvananthapuram.

The current price of a specific AI model API.

The personal biography of the developer using this document.

The current president of any country.

Any fact that is not stated anywhere in this PDF.

Suggested Lesson 19 Test Sequence

1. Ask: What is RAG?

2. Ask: Why are documents split into chunks?

3. Ask: What is the role of ChromaDB?

4. Ask: Why are embeddings useful for semantic search?

5. Ask: What is chunk overlap?

6. Ask: What is a hallucination?

7. Ask: What is the difference between retrieval and generation?

8. Ask an out-of-document question: What is the current weather in Thiruvananthapuram?

9. Change n_results from 3 to 5 to 10 and inspect the retrieved chunks.

10. Print the retrieved chunks and compare them with the final answer.

Note: This is a deliberately self-contained practice knowledge base. The out-of-document questions are included so the RAG application can be tested for grounded 'insufficient information' behavior.